UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Ingeniería en Sistemas de Información

DOCUMENTO DE SEGUIMIENTO Y EVIDENCIAS

Sistema de Gestión de Citas para Barbería

Código del proyecto: PF-03

Curso: Programación II

Docente: Ing. David Álvarez

Estudiante: Angel Estuardo Campos Santay

Carné: 9941-25-4809

Fecha: 8 de noviembre de 2026

Enlace del repositorio

Repositorio de GitHub: https://github.com/Angel311Cam/barbershop-PF03

Definición breve del problema

Muchas barberías todavía organizan sus citas de forma manual. Esto puede causar confusiones con los horarios, duplicación de reservaciones y dificultad para llevar el control de los clientes y los servicios. Para atender esta situación se desarrolló un sistema web que permite consultar los servicios disponibles, registrar citas y administrarlas desde un panel. La información se almacena en PostgreSQL para poder consultarla y mantenerla organizada.

Diagrama de entidades

El diagrama muestra la estructura del sistema de gestión de citas para barbería y cómo se relacionan sus diferentes componentes. Se encuentran los controladores CitaController y ServicioController, encargados de recibir y gestionar las solicitudes; las clases de servicio CitaService y ServicioService, donde se realizan las operaciones y validaciones; y los repositorios CitaRepository y ServicioRepository, que permiten consultar y administrar la información almacenada en la base de datos.

También se muestran las entidades principales Cita y Servicio, con sus respectivos atributos y métodos. La entidad Cita almacena los datos del cliente, teléfono, fecha, hora, precio, duración y estado de la reservación, además de mantener una relación con Servicio, donde se guarda el nombre, descripción, precio, duración y disponibilidad de cada servicio.

Finalmente, se incluye el enumerado Estado, que define los posibles estados de una cita: PROGRAMADA, ATENDIDA y CANCELADA.

Este diagrama permite comprender cómo se organiza el proyecto y cómo se conectan sus componentes desde la recepción de una solicitud hasta el almacenamiento de los datos.

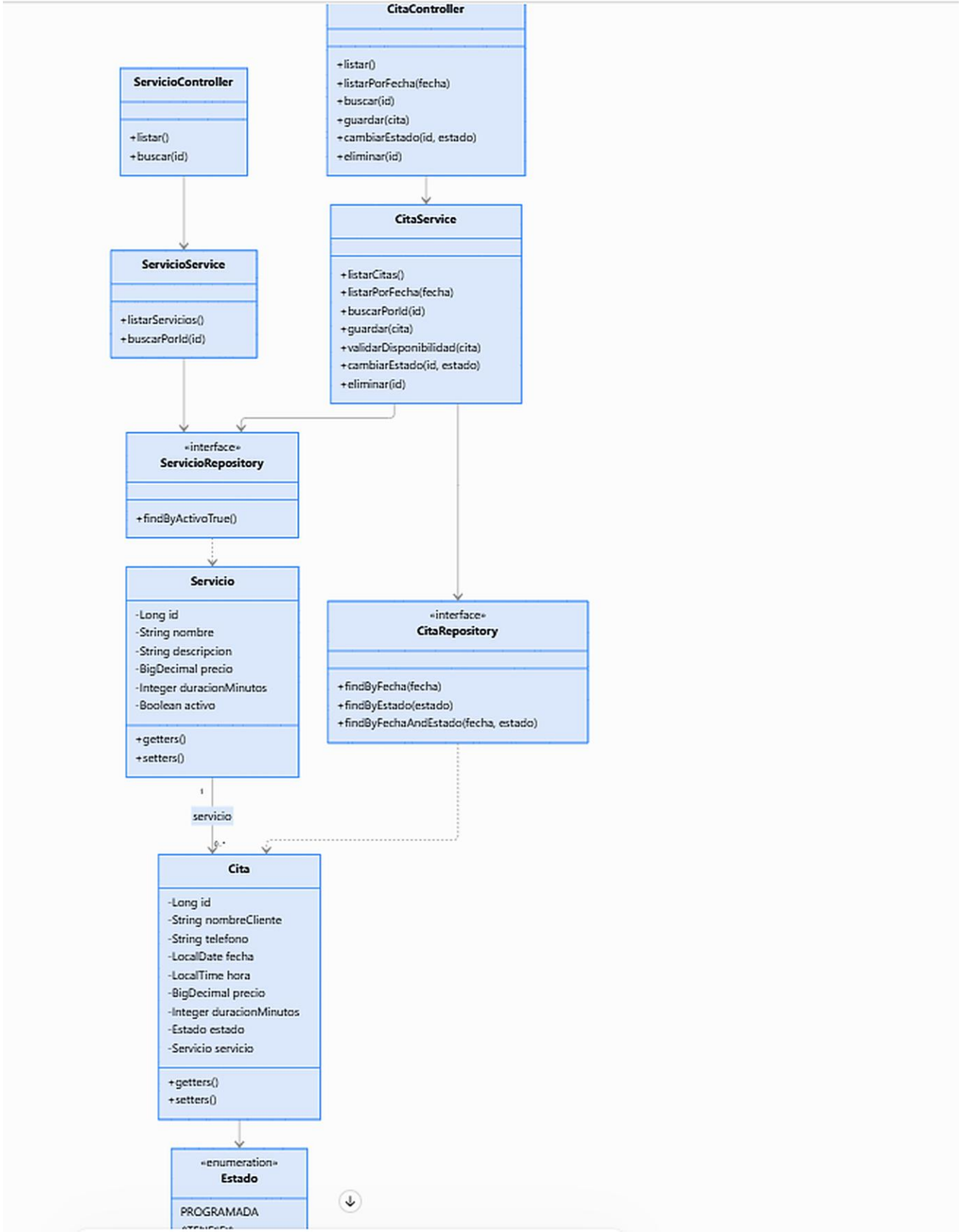


Figura 1. Diagrama de entidades del sistema de gestión de citas para barbería.

1. Evidencias de conexión con PostgreSQL

En esta captura se observa la base de datos barbershop_db abierta en pgAdmin 4 y el uso de Query Tool. Esto permite comprobar que PostgreSQL está disponible para consultar la información del proyecto.

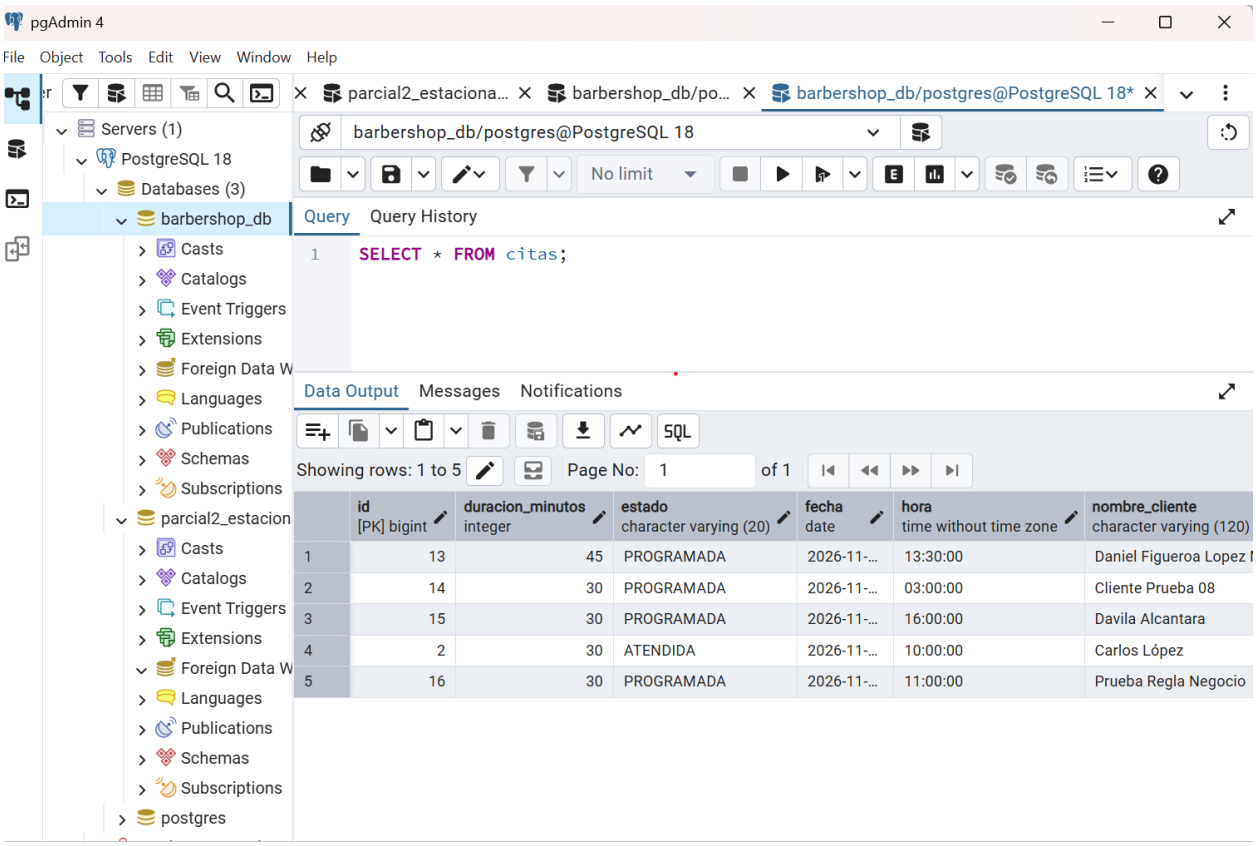


Figura 2. Base de datos barbershop_db y consulta desde pgAdmin 4.

2. Capturas de las funciones terminadas

El backend se organizó en controladores y servicios para separar las solicitudes que recibe la API de las operaciones que trabajan con los datos.

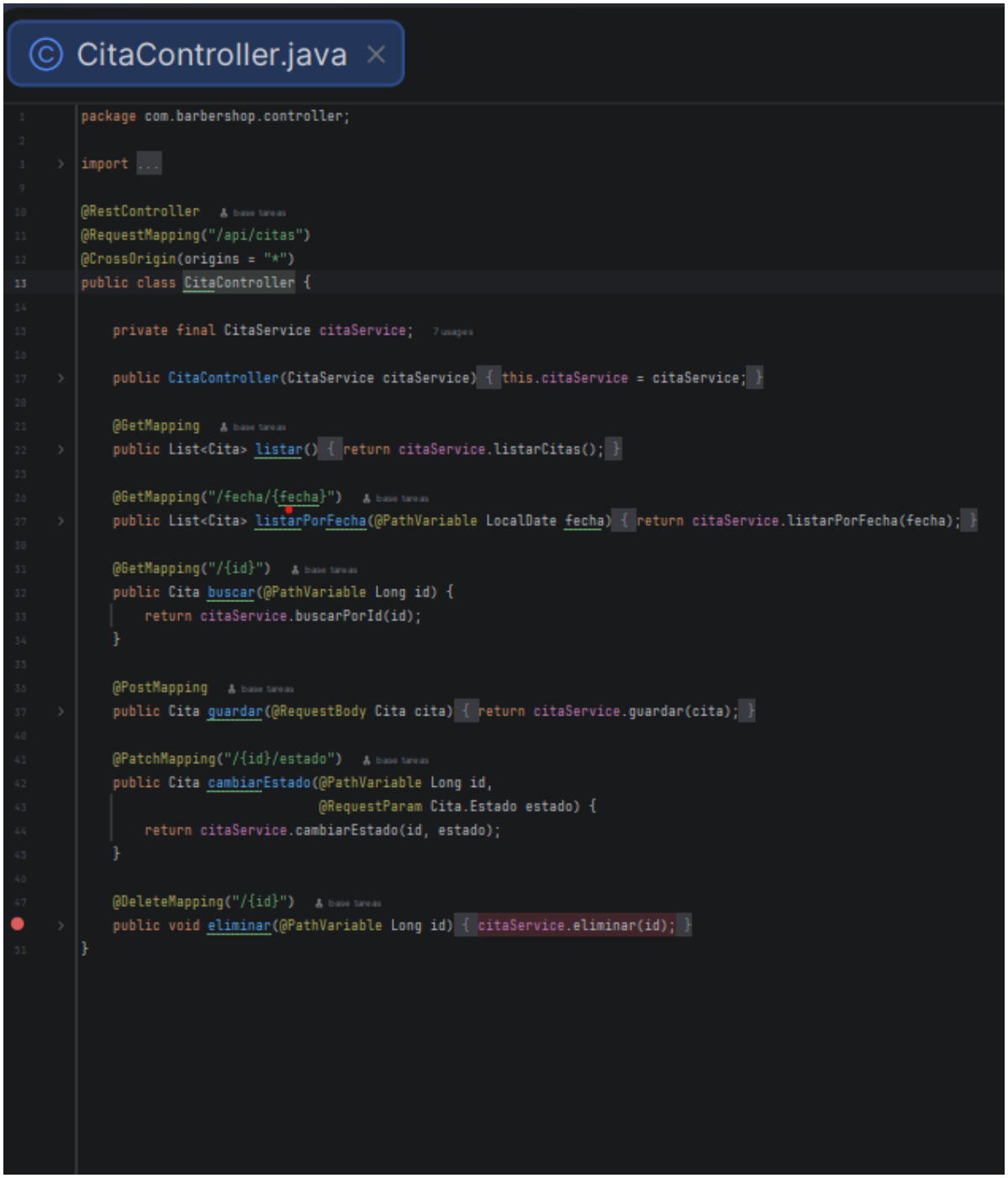


Figura 3. Controlador de citas con sus rutas para consultar, registrar y cambiar estados.



Figura 4. Controlador que dirige la ruta principal hacia la página de inicio.

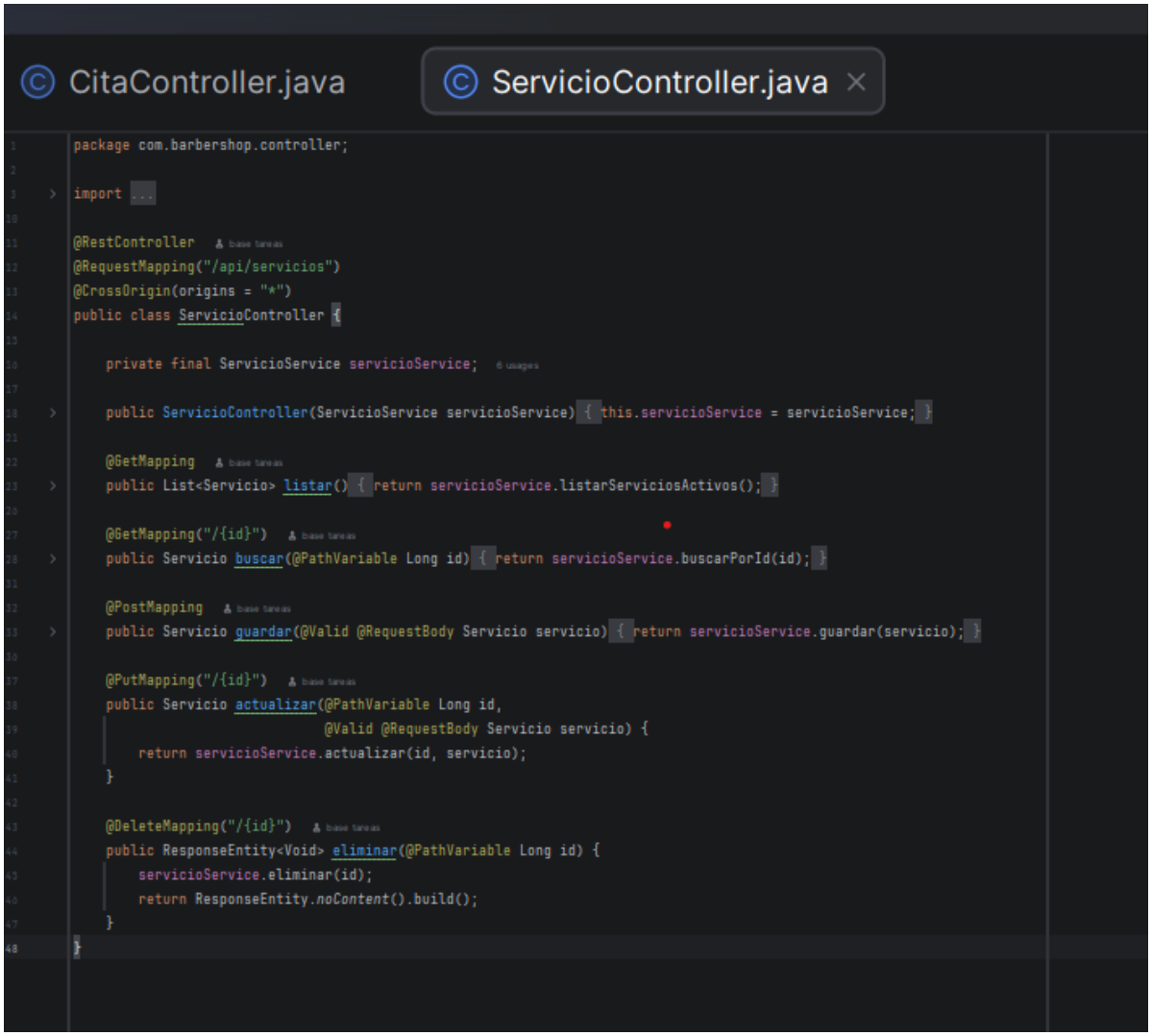


Figura 5. Controlador de servicios para las operaciones de consulta y mantenimiento.



Figura 6. Servicio de citas con las operaciones principales y la validación de disponibilidad.



Figura 7. Servicio encargado de administrar la información de los servicios.

Pruebas realizadas en Postman

Se utilizó Postman para enviar solicitudes a la API y comprobar que los endpoints respondieran. Se realizaron consultas de citas y servicios, el registro de una nueva cita y el cambio de estado de una reservación.

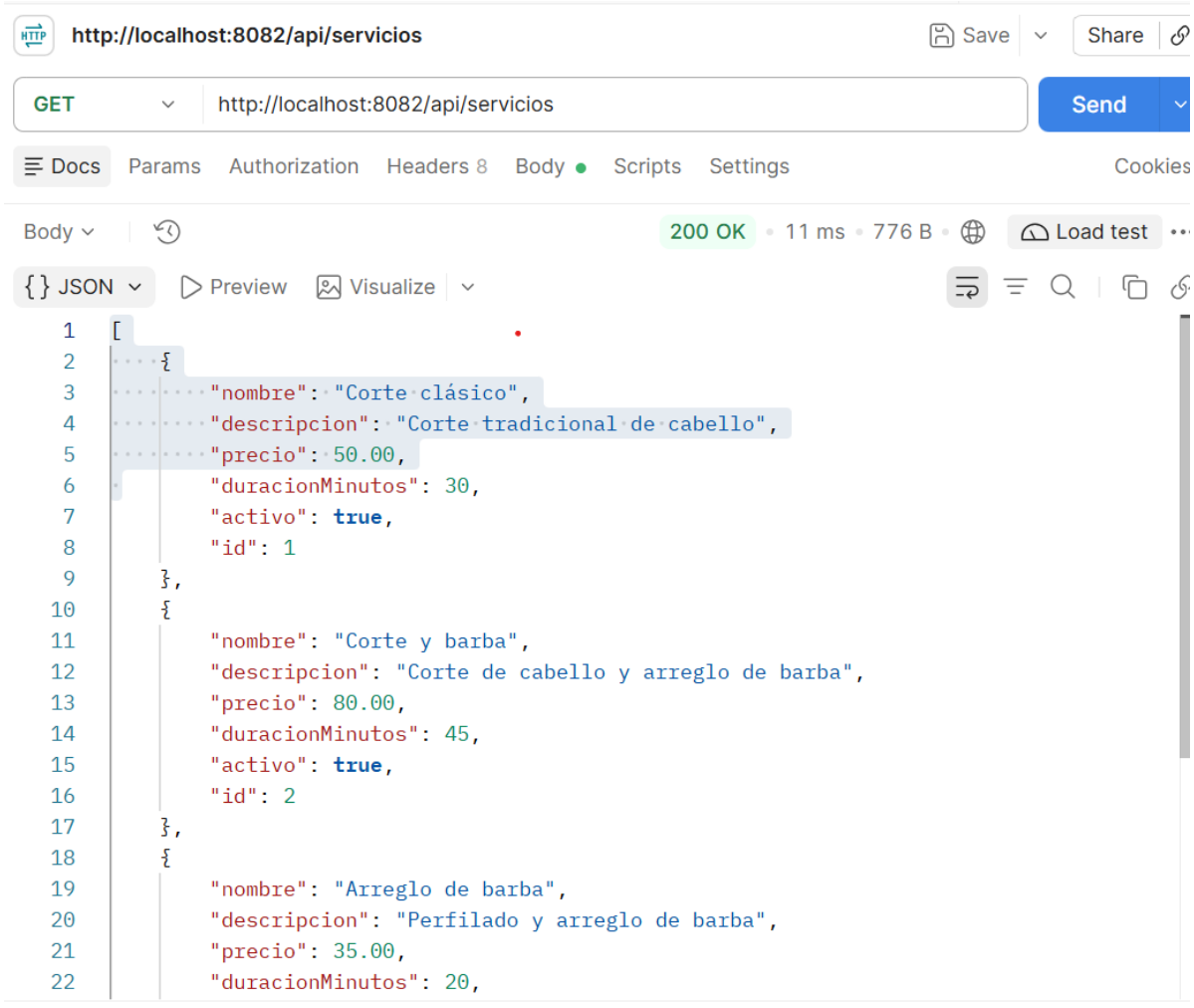


Figura 8. Consulta general de citas mediante GET.

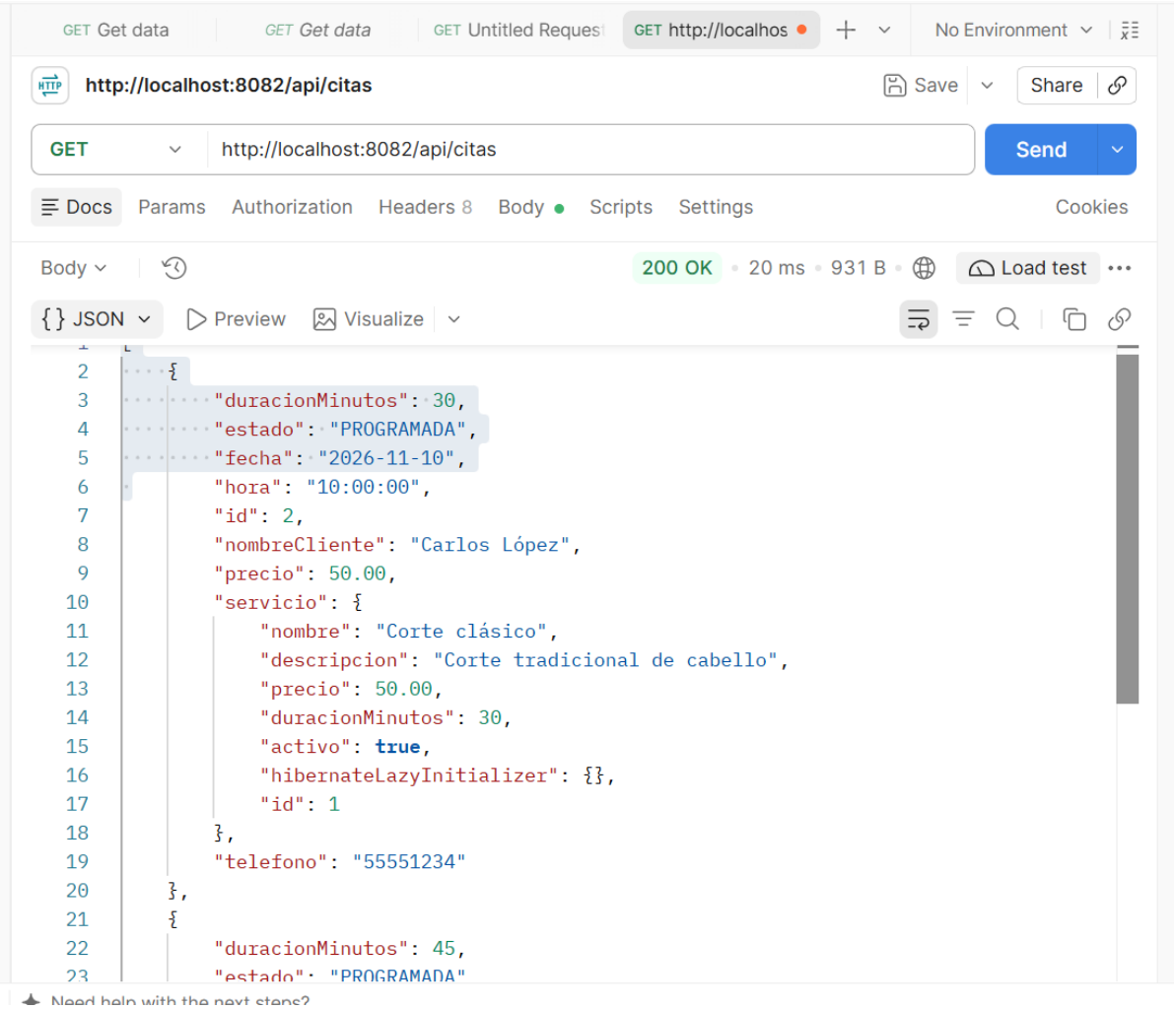


Figura 9. Respuesta de la API con información de las citas registradas.

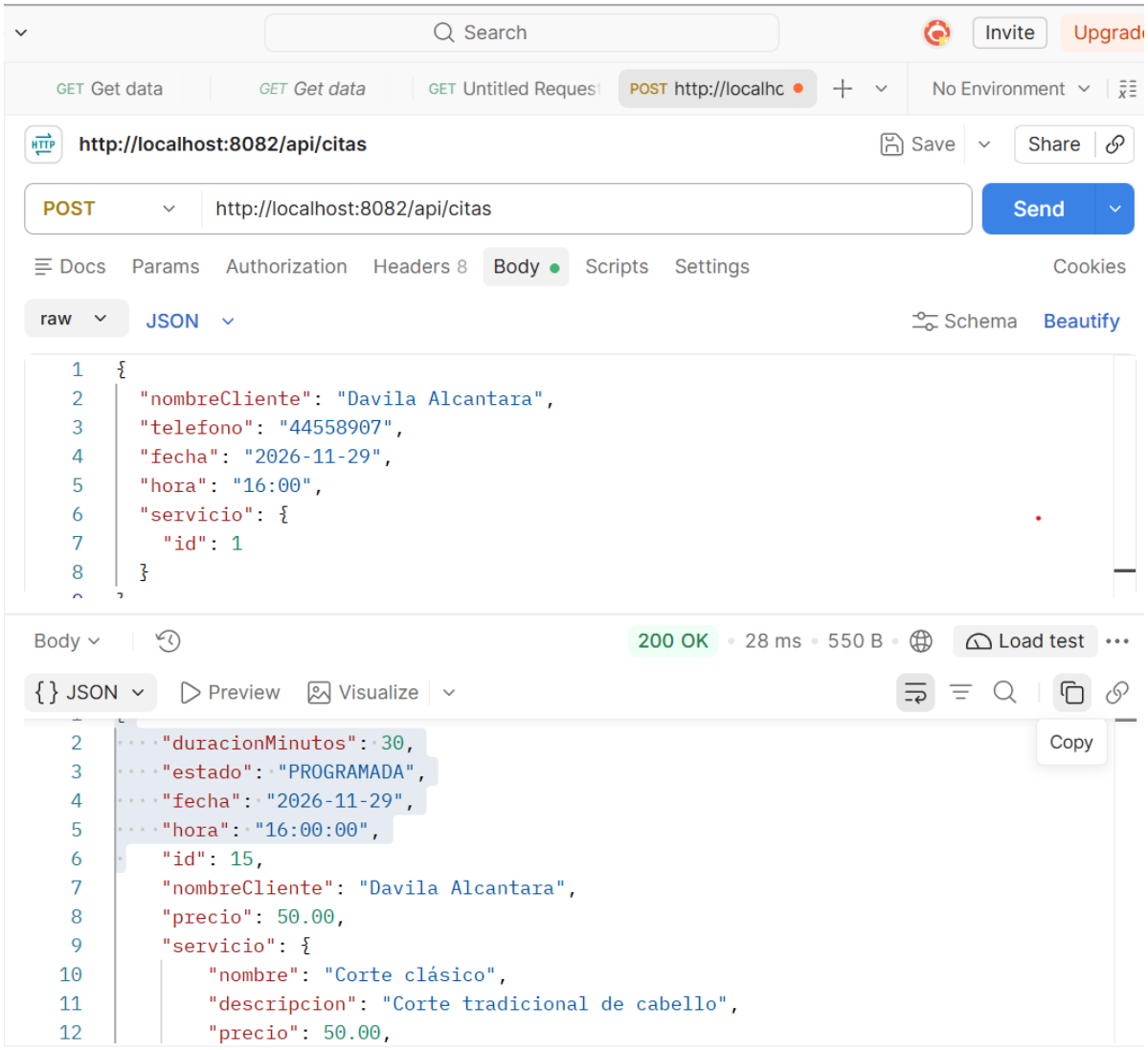


Figura 10. Registro de una cita mediante POST.

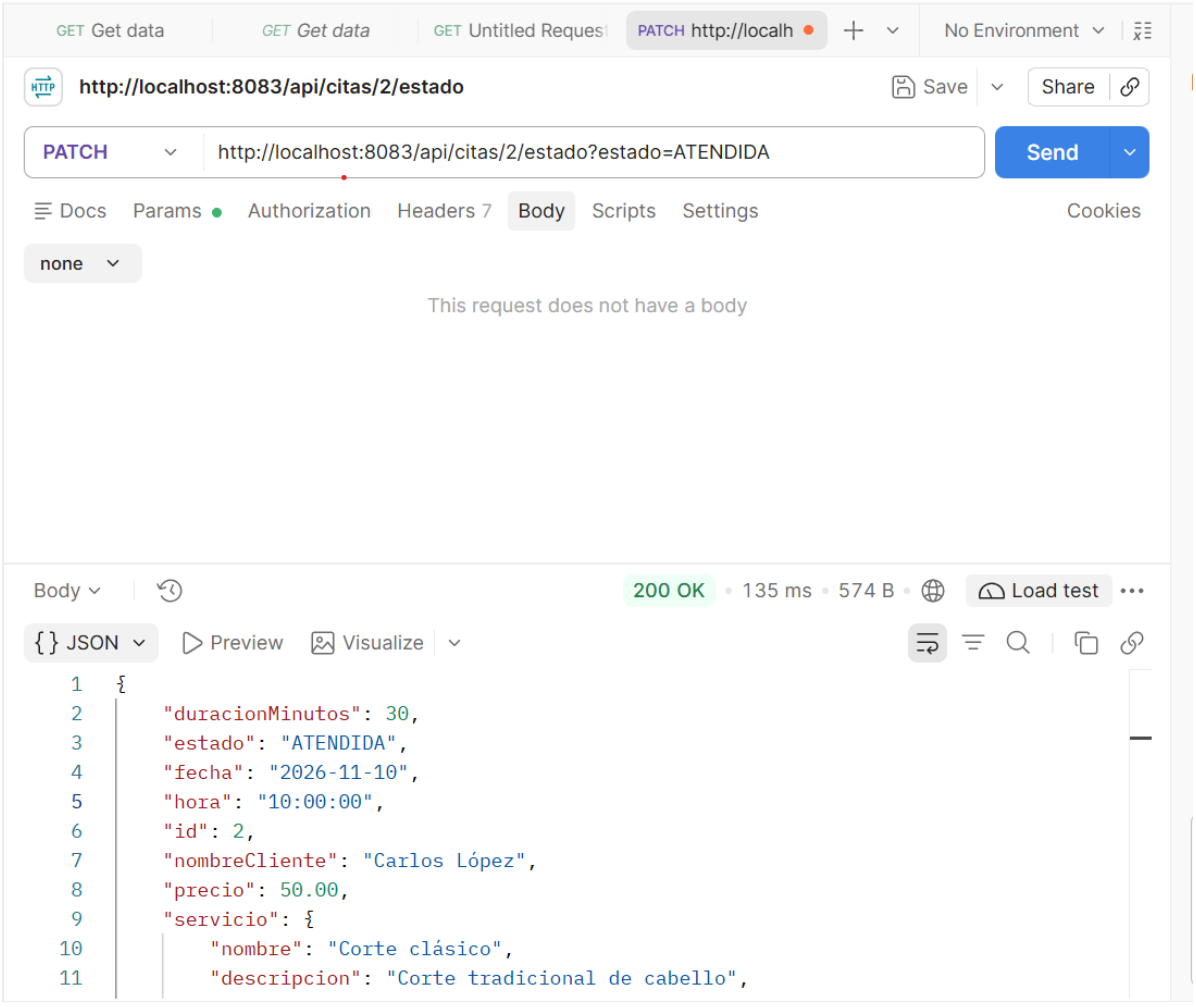


Figura 11. Cambio del estado de una cita mediante PATCH.

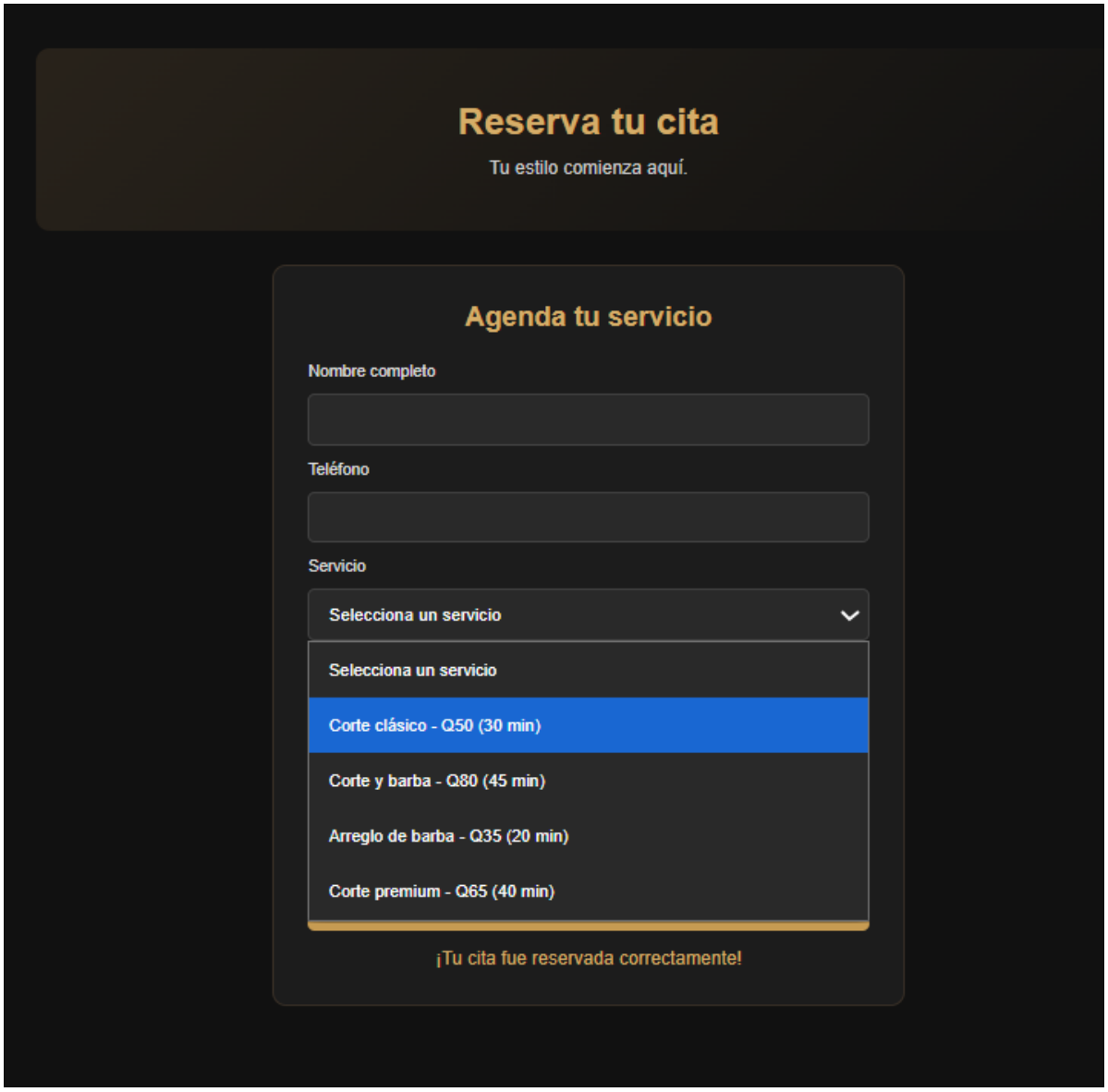


Figura 12. Consulta de los servicios disponibles mediante GET.

Pruebas de validaciones

Se intentó registrar una cita en un horario que ya estaba ocupado. La API respondió con el código 409 Conflict, por lo que no permitió guardar otra reservación que se cruzara con la existente. Esta prueba ayuda a comprobar que el sistema toma en cuenta la duración de los servicios al revisar la disponibilidad.

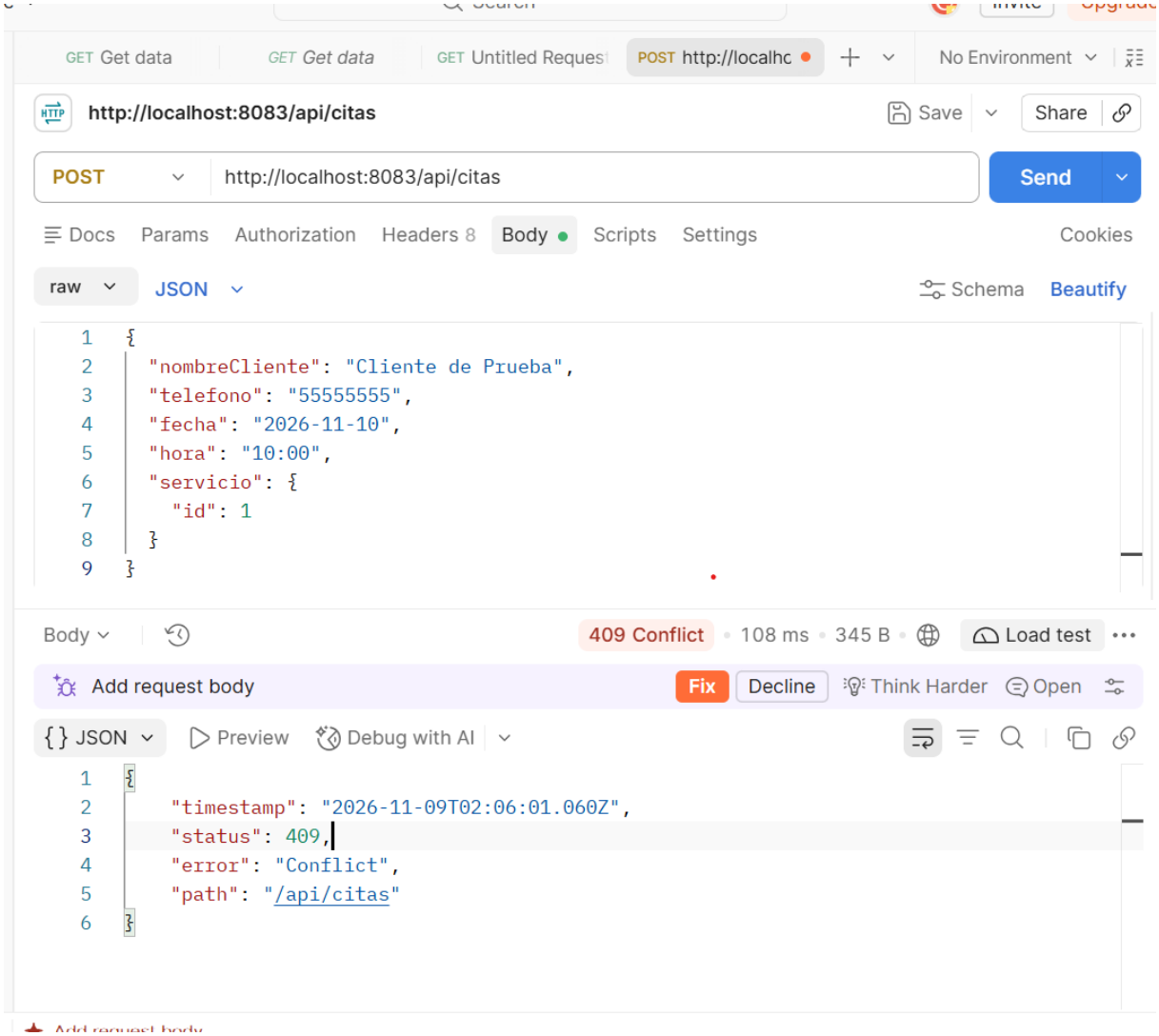


Figura 13. Respuesta 409 Conflict al intentar utilizar un horario ocupado.

Pruebas de excepciones

También se consultó una cita utilizando un identificador que no existe. El sistema respondió con un error 500 Internal Server Error. La prueba permitió comprobar que la búsqueda no devuelve una cita inexistente. Como mejora futura, sería conveniente manejar este caso con una respuesta 404 Not Found para indicar de forma más precisa que el registro no existe.

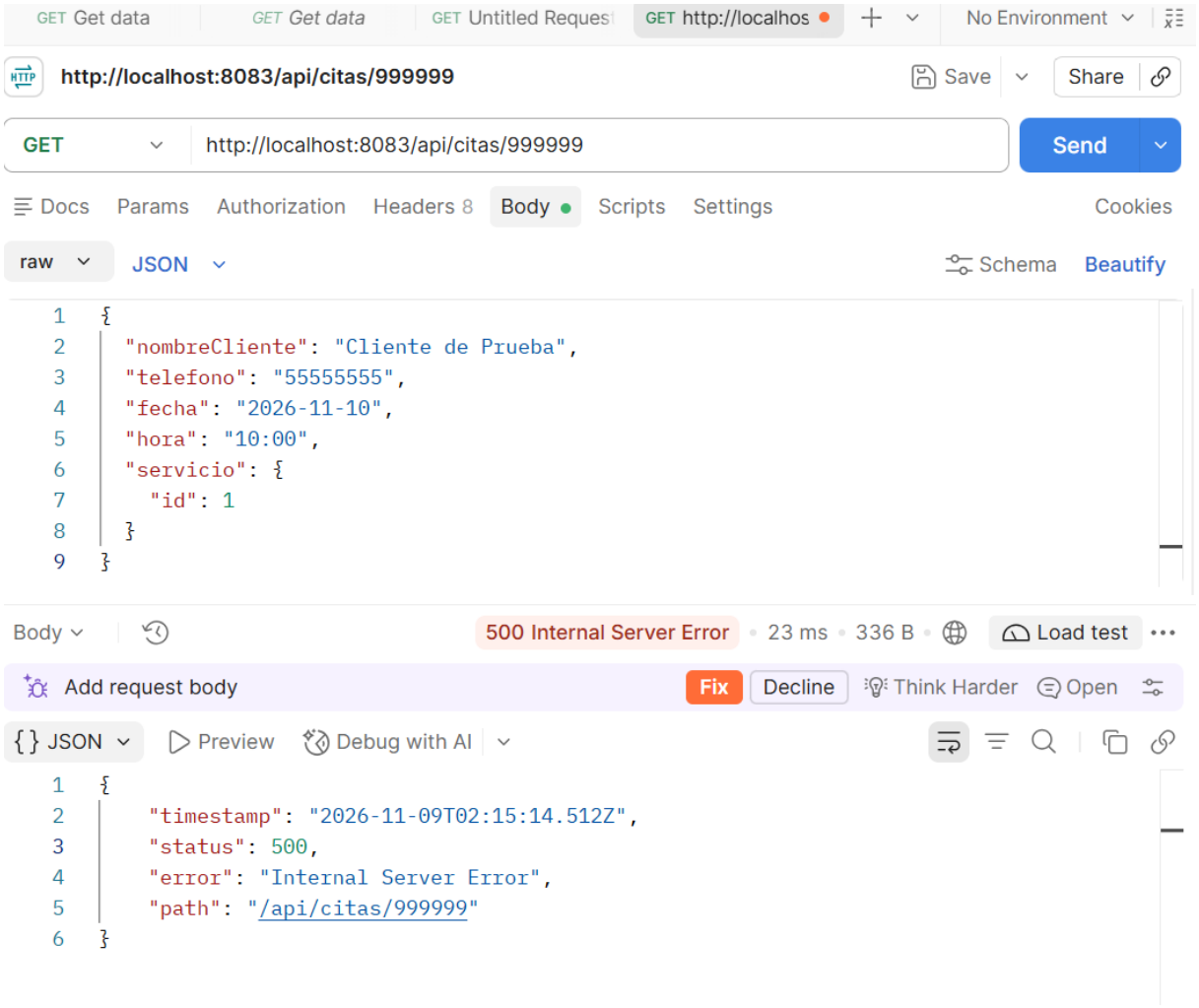


Figura 14. Consulta de una cita con un identificador inexistente.

Pruebas de las reglas del negocio

Se registró una cita seleccionando el servicio Corte clásico. La respuesta mostró que el sistema asignó el precio de Q50.00, una duración de 30 minutos y el estado PROGRAMADA. Estos datos se obtienen del servicio seleccionado, por lo que no fue necesario enviarlos manualmente en el cuerpo de la solicitud.

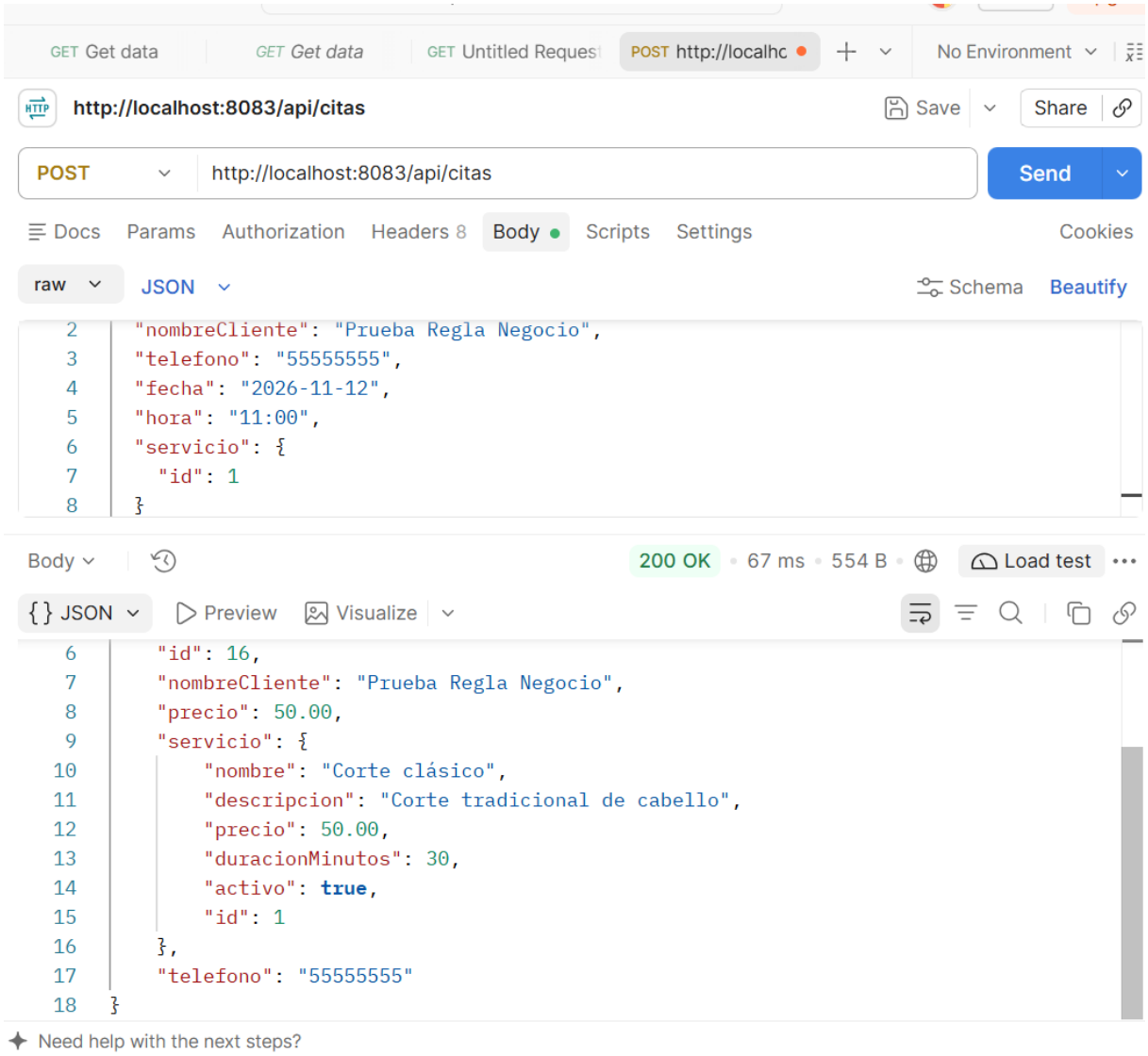


Figura 15. Registro de cita y asignación automática de precio, duración y estado.

Pruebas del frontend

Se revisó la página principal para comprobar que mostrara el formulario de reservación y los campos necesarios para registrar una cita. También se verificó el panel de administración, desde donde se pueden consultar las citas y revisar algunos datos generales.

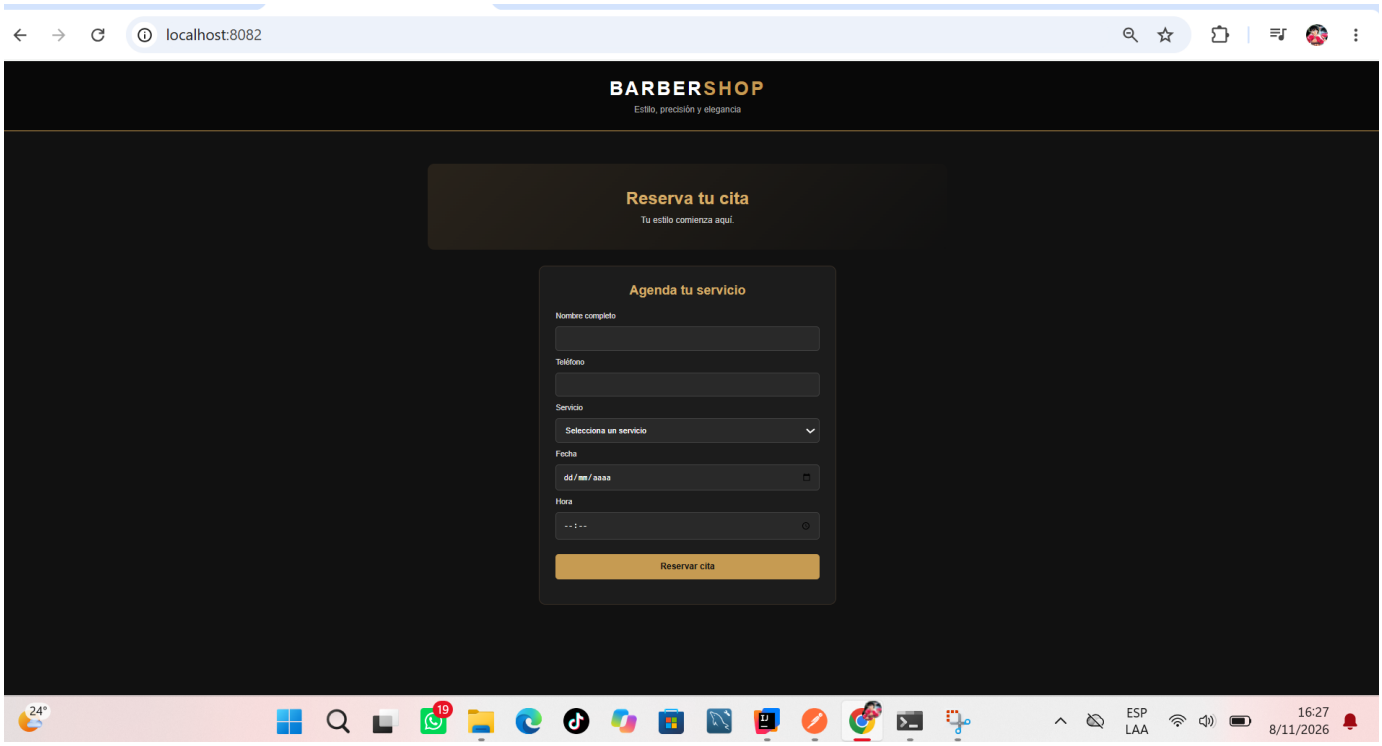


Figura 16. Página principal del sistema con el formulario para reservar una cita.

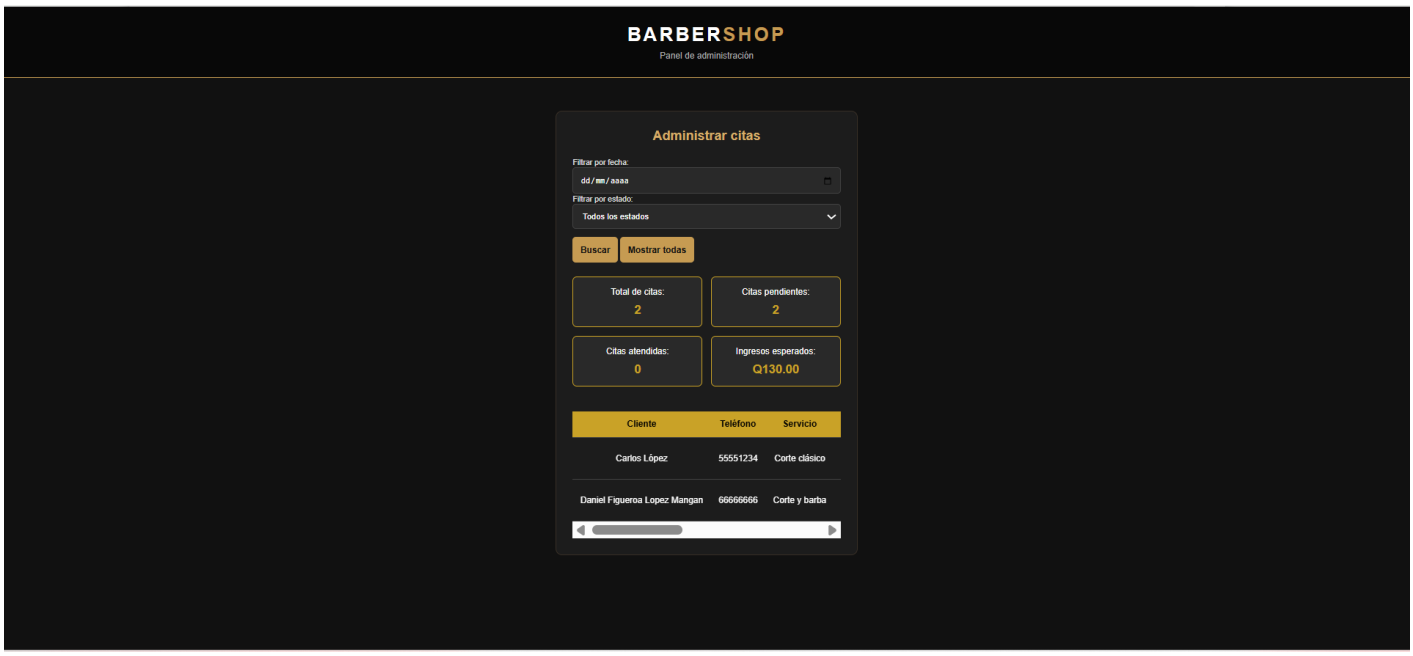


Figura 17. Panel de administración de citas.

Consultas de datos almacenados en PostgreSQL

Se ejecutó la consulta SELECT * FROM citas; desde pgAdmin 4. En el resultado se observan registros guardados en la tabla, incluyendo el identificador, la duración, el estado, la fecha, la

hora y el nombre del cliente. Esto permite comprobar que la información registrada por el sistema queda almacenada en PostgreSQL.

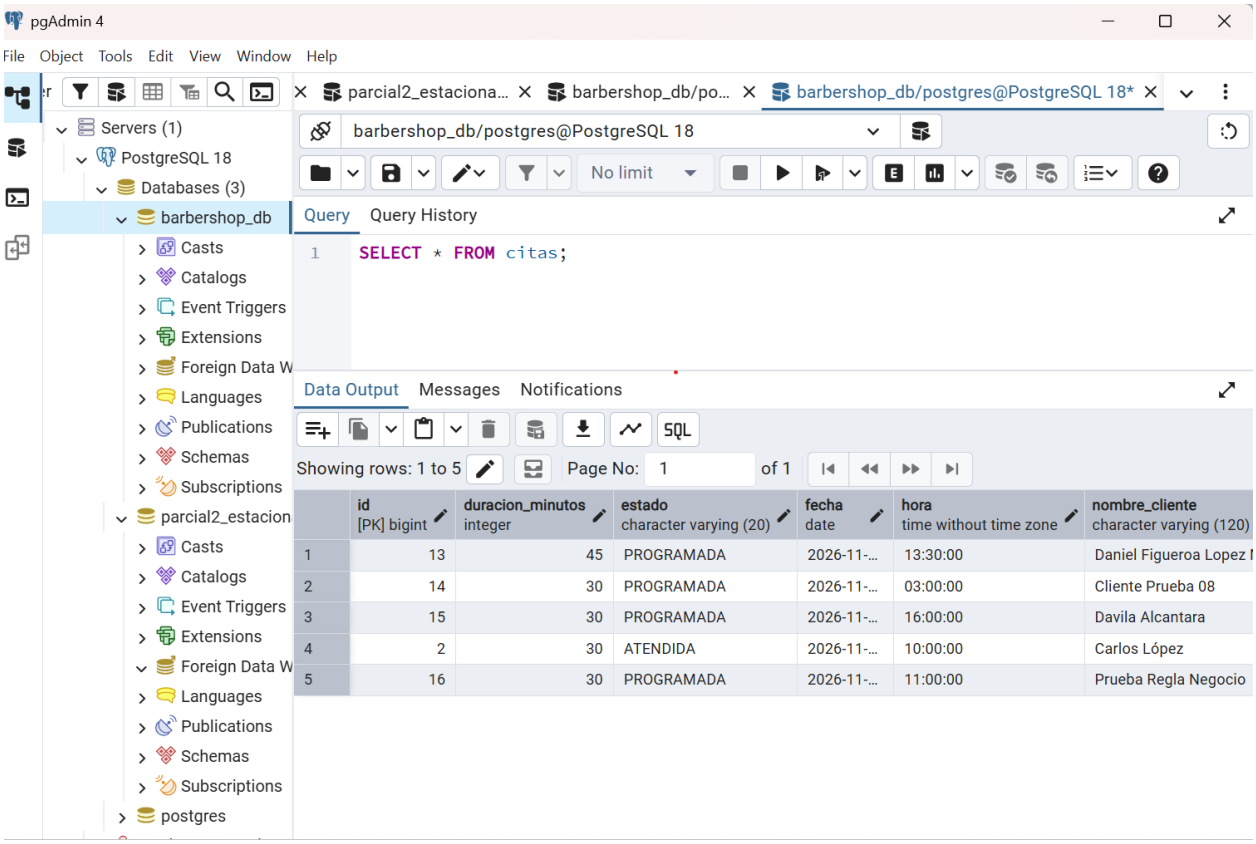


Figura 18. Resultado de la consulta de los registros almacenados en la tabla citas.

Reflexión final

Durante el desarrollo de este proyecto pude practicar varios temas que hemos visto en Programación II. Trabajar con Java, Spring Boot y PostgreSQL me ayudó a entender mejor cómo se conectan las diferentes partes de un sistema y cómo se guarda la información para después consultarla.

Una parte importante fue realizar las pruebas con Postman, porque así pude revisar si las rutas respondían correctamente y detectar situaciones que necesitaban validación, como intentar reservar un horario que ya estaba ocupado. También aprendí que no basta con que el programa funcione en un solo caso, sino que hay que probar diferentes situaciones.

En general, el proyecto me permitió tener una experiencia más completa de desarrollo, desde la base de datos y el backend hasta la interfaz web. Durante el proceso aparecieron algunos errores, pero al ir revisándolos pude comprender mejor el funcionamiento del sistema y mejorar mis conocimientos.